

Experiment 10

Dockerfile is a text file that contains a set of instructions used to automatically build a Docker image. It defines the base image, environment, dependencies, and commands required to run an application inside a container.

1. Dockerfile Overview

A Dockerfile consists of step-by-step instructions that Docker follows to create an image. Each instruction in the Dockerfile creates a new layer in the image.

Common Dockerfile instructions include:

- **FROM:** Specifies the base image
- **RUN:** Executes commands during image build
- **COPY / ADD:** Copies files into the container
- **WORKDIR:** Sets working directory
- **CMD / ENTRYPOINT:** Defines default command to run

2. Pulling an Image from Docker Hub

The command:

```
docker pull hello-world
```

is used to download an image from Docker Hub.

Explanation:

- Downloads the hello-world image from the official Docker repository
- If not present locally, Docker fetches it from the remote registry
- The image is stored locally for future use
- Output shows layers being downloaded and image digest

3. Listing Available Docker Images

The command:

`docker images` displays all locally available

Docker images.

Explanation:

- Shows repository name, tag, image ID, size, and creation details
- Confirms successful image download
- Helps manage and identify images stored in the system

4. Building a Docker Image

A Docker image is built using the

command: `docker build -t <image_name> .`

Explanation:

- Builds an image from the Dockerfile in the current directory
- `-t` is used to assign a name (tag) to the image
- Docker processes each instruction in the Dockerfile step-by-step
- Final output is a reusable image

5. Running a Docker Container

The command:

`docker run hello-world` is used to run a

container from the specified image.

Explanation:

- Docker checks if the image is available locally

- If not, it pulls the image from Docker Hub
- Creates a new container from the image
- Executes the default command defined in the image
- Displays output in the terminal

6. Working of Hello-World Container

When the hello-world container is executed, the following steps occur:

- Docker client sends request to Docker daemon
- Docker daemon pulls the image (if not available locally)
- A container is created from the image
- The container runs a simple program that prints a message
- Output is sent back to the terminal

This confirms that Docker is installed and functioning correctly.

7. Image Storage and Layers

Docker images are built using a layered architecture:

- Each instruction in a Dockerfile creates a layer
- Layers are cached to improve performance
- Images are stored in the local Docker registry

Conclusion

Thus, Dockerfile provides a structured way to define application environments and build Docker images. Commands like `docker pull`, `docker images`, `docker build`, and `docker run` help in managing images and running containers. The successful execution of the hello-world container confirms proper Docker setup and understanding of image execution workflow.

```
C:\Users\15L>docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview hello-world

C:\Users\15L>
```

```
C:\Users\15L>docker images

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
hello-world:latest   1b44b5a3e06a      10.1kB        0B
httpd:latest          4613a77dcb46      117MB         0B              U
mysql:latest          a88c3e85e887      632MB         0B
nginx:1.0             fcbd66995946      169MB         0B              U
node:latest           c3978d05bc68      1.1GB         0B              U
redis:latest          170a1e90f843      138MB         0B
ubuntu:latest         ca2b0f26964c      77.9MB        0B              U

C:\Users\15L>
```

```
C:\Users\15L>docker run hello-world
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

```
C:\Users\15L>
```

```
C:\Users\15L>docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
f60ff2c3de0b   hello-world    "/hello"                About a minute ago    Exited (0) About a minute ago
0d918080eacd2   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
e81c8ac6cd66   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
f90bf8584ef0   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
c1558a5bdfc7   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
61c302b4db0a   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
a916b1bc1d13   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
004bcde709ff   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
c0cb01650812   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
d60947f1917e   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
< 23a0452c     httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
o8ef6efe       httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
74b68ec74672   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
55a80be3d445   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
4ed8cfaded6b   httpd         "httpd-foreground"     4 months ago    Exited (0) 4 months ago
2fa633478597   ubuntu       "/bin/bash"            23 months ago    Exited (255) 6 months ago
ee37470b71cd   node         "docker-entrypoint.s..." 24 months ago    Exited (0) 23 months ago
9c7ca0012691c   node         "docker-entrypoint.s..." 24 months ago    Exited (0) 23 months ago
f4b5096c1f14   d1397258b209 "docker-entrypoint.s..." 24 months ago    Exited (255) 6 months ago
d820ef6a1a06   d2c94e258dcb "/hello"           24 months ago    Exited (0) 24 months ago
bb71b1a51783   d2c94e258dcb "/hello"           24 months ago    Exited (0) 24 months ago
922a7dec7144   d1397258b209 "docker-entrypoint.s..." 24 months ago    Exited (255) 24 months ago
97abe06c14c7   d2c94e258dcb "/hello"           24 months ago    Exited (0) 24 months ago
a67130964a00   d2c94e258dcb "/hello"           24 months ago    Exited (0) 24 months ago
b0f4789574a5   d2c94e258dcb "/hello"           24 months ago    Exited (0) 24 months ago
a21892131937   nginx:1.0     "/usr/sbin/nginx -g ..." 1 years ago      Exited (255) 24 months ago
0.0.0.0:9090->80/tcp
webserver
```